
Software Requirements Specification

for

LLMUSE

Version 1.0 approved

Prepared by Team 4

2025. 11. 09.

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction	1
1.1 Purpose	1
1.2 Scope	
1.3 Definitions, Acronyms, Abbreviations	
1.4 References	
1.5 Overview	
2. Overall Description	2
2.1 Product Perspective	2
2.1.1 System Interfaces	
2.1.2 User Interfaces	
2.1.3 Hardware Interfaces	
2.1.4 Software Interfaces	
2.1.5 Memory Constraints	
2.1.6 Operations	
2.2 Product Functions	3
2.2.1 데이터 수집	
2.2.2 벡터 DB 구축	
2.2.3 RAG	
2.2.4 LLM prompt	
2.2.5 LLMUSE 메인페이지	
2.3 User Classes and Characteristics	3
2.3.1 User	
2.3.2 System Administer	
2.4 Design and Implementation Constraints	3
2.4.1 Data constraints	
2.4.2 External API constraints	
2.4.3 Security constraints	
2.4.4 LLM prompt	
2.4.5 Performance constraints	
2.5 Assumptions and Dependencies	4
2.5.1 Internet, external API dependencies	
2.5.2 데이터베이스 구축 선행 가정	
2.5.3 데이터 매칭 가정	
2.5.4 외부 API 응답 형식 동일 가정	
3. External Interface Requirements	5
3.1 External Interfaces	5
3.1.1 User Interfaces	
3.1.2 Hardware Interfaces	
3.1.3 Software Interfaces	
3.1.4 Communication Interfaces	
3.2 Function Requirements	8
3.2.1 Use Case	
3.2.2 Use Case Diagram	
3.2.3 Data Dictionary	
3.3 Performance Requirements	9
3.3.1 Static Numeric Requirements	
3.3.2 Dynamic Numeric Requirements	
3.4 Logical Database Requirements	10

3.4.1 Type of data	
3.4.2 Frequency of use	
3.4.3 Access control	
3.4.4 Integrity constraints	
3.5 Design Constraints	11
3.5.1 Physical design constraints	
3.5.2 User class	
3.6 Software System Attributes	11
3.6.1 Reliability	
3.6.2 Availability	
3.6.3 Security	
3.6.4 Maintainability	
3.6.5 Portability	

Revision History

Name	Date	Reason For Changes	Version
SRS	25.11.09.	First Version	1.0

1. Introduction

1.1 Purpose

이 문서는 일반 유저를 대상으로 영화 추천 및 정보를 안내하는 LLM 기반 웹 프로젝트의 요구사항을 명확히 하고, 추후 유지보수 하는 과정에서 참고할 수 있도록 제작되었다.

1.2 Scope

LLMUSE는 LLM(Large Language Model)을 활용한 영화 추천 및 정보 제공 웹 애플리케이션이다. 이 시스템은 RAG(Retrieve-Augment-Generate) 구조를 채택하여 사용자의 자연어 질문을 분석하고, 벡터 데이터베이스 검색 및 LLM의 자연어 이해를 결합하여 영화 추천 및 정보를 제공한다. 주요 기능은 KOBIS 및 TMDB API를 활용한 데이터 수집, ChromaDB를 사용한 벡터 DB 구축, RAG 기반 답변 생성, LLM 프롬프트 엔지니어링, 메인 페이지의 챗봇 인터페이스를 통한 질의응답 기능을 포함한다.

1.3 Definitions, Acronyms and Abbreviations

LLM : Large Language Model

RAG : Retrieve-Augment-Generate

KOBIS : 영화진흥위원회에서 제공하는 영화관입장권통합전산망(KOREA Box-office Information System, KOBIS)이다. 전국 영화관의 입장권 발권정보를 실시간으로 집계처리하여 다양한 데이터를 제공한다.

TMDB API : The Movie DataBase(TMDB)에서 제공하는 open API로, 영화의 다양한 정보들을 제공한다.

1.4 References

IEEE Std 830-1998 IEEE Recommended Practice for Software Requirements Specifications, In IEEEExplore Digital Library

<http://ieeexplore.ieee.org/Xplore/guesthome.jsp>

The Movie Database(TMDB) API Documentation

<https://developer.themoviedb.org/docs>

1.5 Overview

본 소프트웨어 요구사항 명세서는 네 챕터로 구성되어 있다. 첫번째 챕터에서는 LLMUSE, 영화 추천 서비스와 그 목적에 대한 소개 및 본 문서에서 사용하는 약어 및 용어에 대한 설명을 하고 있다. 두번째 챕터에서는 시스템 인터페이스 및 기능, 다른 시스템과의 상호작용을 포함한 product perspective에 대한 전반적인 설명을 제공한다. 세번째 챕터에서는 외부 인터페이스, 기능 등을 포함한 자세한 요구사항 명세화를 진행한다.

2. Overall Description

2.1 Product Perspective

LLMuse는 LLM(Large Language Model)을 활용한 영화 추천 및 정보 제공 웹 애플리케이션이다. 이 시스템은 RAG(Retrieve-Augment-Generate) 구조를 채택하여 작동한다. 사용자 질문을 자연어로 입력 받으면, 시스템은 이를 분석한 뒤 벡터 데이터베이스를 활용한 의미 기반 검색을 통해 관련 영화 데이터를 검색한다. 이 검색 결과는 LLM의 자연어 이해 능력과 결합되어 사용자의 질문 의도를 정확히 파악하고 최적의 영화 추천 및 정보를 제공한다.

2.1.1 System interfaces

사용자는 웹 브라우저를 통해 프론트엔드(Frontend)와 상호작용하며 자연어 질의를 입력하고 응답을 수신한다. 프론트엔드와 백엔드(Backend)는 RESTful API를 통해 질의와 응답을 교환하는 핵심 게이트웨이 역할을 수행한다. 백엔드는 영화 데이터 검색을 위해 ChromaDB와 연동하여 벡터 검색 쿼리를 전송하고, Overview, Title, Director, Actor의 네 가지 주요 컬렉션으로부터 의미 기반 데이터를 검색한다. 또한 LLM Service(OpenAI API)를 통해 사용자 질문의 의도 분석과 답변 생성을 수행하며, 최신 박스오피스 정보를 수집하기 위해 KOBIS API, 영화 상세 정보를 가져오기 위해 TMDB API와 상호작용한다.

2.1.2 User interfaces

사용자 인터페이스는 웹 브라우저 기반의 반응형 웹 디자인으로, 데스크톱 환경에 최적화된 UI/UX를 제공한다. 핵심 구성은 실시간 대화형 챗봇(chat-bot) 인터페이스로, 사용자는 텍스트 입력창을 통해 자연어 질의를 입력할 수 있다. 응답은 자연어 기반의 텍스트 형태로 제공된다.

2.1.3 Hardware interfaces

본 시스템은 웹 애플리케이션 형태로 동작하며, 데스크톱, 또는 노트북(1.5GHz 이상 CPU, 2GB 이상 RAM) 환경에서 사용 가능하다. 입력 장치는 키보드, 마우스이며, 출력 장치는 모니터 또는 디스플레이이다.

2.1.4 Software interfaceUser class

본 시스템은 적어도 Windows 7 이상의 OS에서 동작할 수 있으며 Windows 10 및 Windows 11의 환경에 최적화 되어있다. 시스템 사용을 위해서는 Chrome 100.0+, Safari 15+ 등 최신 웹 브라우저와 JavaScript 실행 환경이 필수적이다.

2.1.5 Memory constraints

최소 2GB 이상의 RAM을 철시한 컴퓨터를 권장한다.

2.1.6 Operations

User operations:

사용자는 챗봇 인터페이스를 통해 영화 추천 및 정보 질의를 수행한다. 자연어 입력을 통해 장르, 배우, 감독, 분위기 등 다양한 기준의 영화 추천을 요청하거나, 특정 영화의 상세 정보 및 최신 박스오피스 순위를 확인할 수 있다.

System administrator operations:

관리자는 KOBIS 및 TMDB API를 정기적으로 호출하여 최신 데이터를 수집하고, 영화 제목 매칭 및 검증 절차를 거쳐 ChromaDB 벡터 데이터베이스를 구축, 갱신한다. 또한, LLM Prompt Engineering과 하이퍼파라미터 튜닝을 통해 답변 품질과 일관성을 지속적으로 개선한다. 추가로 API 요청 제한 모니터링, 서버 성능

분석, 보안 점검, 로그 분석 및 오류 추적을 수행하며, 시스템 장애 발생 시에는 기본 캐시 데이터 기반의 제한적 추천 서비스를 제공한다.

2.2 Product Functions

2.2.1 데이터 수집

LLMuse는 한국 사용자에게 최적화된 영화 데이터를 구축하기 위해 KOBIS와 TMDB API를 함께 활용한다. KOBIS API를 통해 2004년부터 2025년까지의 한국 극장 박스오피스 상위권에 집계된 영화를 1차적으로 선별하고, TMDB API를 통해 해당 영화의 상세정보(줄거리, 포스터, 배우, 감독, 키워드 등)을 2차로 수집하여 데이터셋을 확보한다.

2.2.2 벡터 DB 구축

수집된 영화 데이터들은 벡터 DB인 ChromaDB에 저장된다. 각각의 데이터들은 4개의 주요 컬렉션(Overview, Title, Director, Actor)으로 분류되어 벡터화되며, 각 컬렉션은 검색 정확도와 속도 향상을 위해 독립적으로 관리된다. 모든 컬렉션 속 데이터들은 공통적으로 영화의 전체 상세 정보인 Metadata를 포함하며, 각 컬렉션의 목적에 맞는 텍스트들이 벡터화되어 저장된다.

2.2.3 RAG

LLMuse는 RAG 방식을 통해 사용자의 질의에 대한 답변을 생성한다. 사용자의 질문은 LLM을 통해 의미, 의도가 분석되고 그 결과에 따라 가장 관련성이 높은 컬렉션으로 쿼리가 전송된다. 해당 컬렉션에서 벡터 검색과 조건 필터링을 결합하여 가장 적절한 영화 정보를 검색하게 된다. 검색된 영화 정보와 사용자의 원본 질문이 LLM에 전달되고, 최종적으로 사용자에게 보여질 답변을 생성하게 된다.

2.2.4 LLM prompt

LLMuse는 영화 도메인에 특화된 서비스이므로, 하이퍼파라미터 튜닝 및 Prompt Engineering을 통해 답변의 신뢰도, 속도, 문체 일관성 등을 확보하고 개선한다.

2.2.5 LLMUSE 메인 페이지

메인 페이지에서 사용자는 텍스트 입력창을 통해 영화와 관련된 질문을 입력하고, LLMUSE는 실시간으로 이를 분석하여 자연어 기반의 답변을 반환하게 된다.

2.3 User Classes and Characteristics

2.3.1 User

LLMuse 서비스를 직접 이용하는 주체로, 웹 기반 인터페이스를 통해 영화와 관련된 정보를 질의한다. 주로 현재 개봉 중인 인기 영화를 추천 받거나, 특정 영화의 줄거리, 감독, 배우들의 정보를 검색할 수 있다. User는 텍스트 입력창에 자연어로 질문을 작성하면 LLMUSE로부터 실시간 응답을 수신받는다.

2.3.2 System administrator

LLMuse 서비스의 전반적인 관리와 성능 향상을 담당하는 관리자이다. 데이터베이스의 유지보수, API 연동관리, 보안 점검 및 서버 운영을 수행하며, 이를 위해 전반적인 CS 지식 및 LLM을 다뤄본 경험이 필요하며 전체적인 프로젝트 흐름을 파악하고 있어야 한다. LLM Prompt 고도화 및 ChromaDB 구조 최적화를 통해 서비스 응답 속도 및 품질을 개선한다.

2.4 Design and implementation constraints

2.4.1 Data constraints

모든 영화 데이터를 저장하는 것은 현실적으로 불가능하므로, 주요 사용자층(한국인)의 관심사에 맞춰 데이터 수집 범위를 최근 20년간의 한국 극장 박스오피스 상위권 영화를 우선적으로 수집하도록 제한한다.

2.4.2 External API constraints

데이터 수집에 필요한 KOBIS, TMDB API는 하루 및 분당 요청 횟수에 제한이 존재한다. 이에 따라 시스템은 요청 빈도를 초과하지 않도록 로직을 설계해야 한다. 또한 API 호출 실패 시의 재시도 로직 또는 대체 API 사용 등의 방식이 구현되어야 한다.

2.4.3 Security constraints

KOBIS, TMDB, OpenAI 등의 외부 API key는 .env 파일을 통해 환경 변수로 관리되어야 한다. 또한 모든 인증 정보는 암호화 방식으로 저장 및 호출되어야 하며, 외부 로그에 노출되어서는 안된다.

2.4.4 LLM prompt

시스템 개발은 Next.js(Frontend)와 Nest.js(Backend) 프레임워크를 기반으로 한다. 또한 RAG를 사용하기 위해 ChromaDB를 사용한다.

2.4.5 Performance constraints

사용자의 질문에 대한 시스템의 응답 시간은 1분 이내를 목표로 한다. 응답 지연 시에는 오류 메시지 또는 Default 답변을 반환한다.

2.5 Assumptions and dependencies

2.5.1 Internet, external API dependencies

LLMuse의 주요 기능들은 안정적인 네트워크 환경을 전제로 한다. 또한 OpenAI, KOBIS, TMDB API를 외부 서비스로 사용하므로, 해당 API들이 정상적으로 작동해야 한다. 외부 API가 중단되거나 호출 제한이 발생할 경우, 답변 생성 및 데이터 수집 기능이 정상적으로 수행되지 않을 수 있다.

2.5.2 데이터베이스 구축 선행 가정

LLMuse에서 RAG를 활용한 답변을 생성하기 위해 벡터형 데이터베이스(ChromaDB)에 영화데이터가 미리 구축되어 있어야 한다. 데이터가 충분히 확보되지 않거나 최신 영화 정보가 누락된 경우, 시스템의 검색 결과 및 응답 품질이 저하될 수 있다.

2.5.3 데이터 매칭 가정

데이터 수집 과정에서 KOBIS와 TMDB에서 제공하는 “영화제목”은 완벽히 일치한다고 가정한다. 만약 두 데이터 소스 간 제목 불일치가 발생할 경우, TMDB로부터 해당 영화의 상세 정보를 가져올 수 없으며, 일부 데이터가 누락될 가능성이 존재한다.

2.5.4 외부 API 응답 형식 동일 가정

외부 API의 응답 형식(JSON 구조 등)은 일정하게 유지된다고 가정한다. API 응답 형식이 변경된다면, 데이터 파싱 로직이 수정될 필요가 있다.

3. External Interface Requirements

3.1 External interfaces

3.1.1 user Interfaces

이름	메인 홈페이지 화면
목적 / 내용	사용자에게 공연 및 영화 정보를 제공하는 메인 인터페이스
입력 주체 / 출력 목적지	클라이언트 / 사용자
범위 / 정확도 / 허용 오차	범위: 화면에서의 버튼 개수에 따른 입력 범위 정확도: 유저의 마우스 및 키보드 입력에 따른 정확도 허용 오차: 해당 없음
단위	화면
시간 / 속도	사용자의 입력에 따른 화면 전환
타 입출력과의 관계	사용자의 입력을 위한 인터페이스로서 출력 후 사용자의 입력 대기
화면 형식 및 구성	상단 네비게이션 바: 로고, 검색창, 사용자 프로필 메뉴 메인 배너: “오늘의 추천 공연 / 영화” 캐러셀 카테고리 섹션: 공연, 영화, 뮤지컬, 연극 탭 AI 추천 섹션: 개인화된 콘텐츠 그리드 하단 푸터: 서비스 정보 및 이용 약관
윈도우 형식 및 구성	반응형 레이아웃: 데스크톱($\geq 1280\text{px}$), 태블릿($768\text{px} - 1279\text{px}$), 모바일($<768\text{px}$) Next.js 기반 서버 사이드 렌더링 Tailwind CSS 및 Material-UI 컴포넌트 활용
데이터 형식 및 구성	JSON(공연, 영화 메타 데이터), 이미지(포스터) 텍스트(설명)
명령 형식	버튼 클릭, 검색어 입력, 필터 선택
종료 메시지	해당 없음

Table 3.1: User Interface of Main Page

이름	검색 결과 화면
목적 / 내용	사용자가 검색한 공연/영화에 대한 결과를 표시
입력 주체 / 출력 목적지	사용자 / 서버
범위 / 정확도 / 허용 오차	검색어에 따른 정확한 결과 반환, 오차 허용 범위 없음
단위	화면
시간 / 속도	검색 요청 후 2초 이내 결과 표시
타 입출력과의 관계	검색 입력 후 백엔드 API 호출 및 결과 수신
화면 형식 및 구성	검색 필터 사이드바: 장르, 날짜, 지역, 가격대 결과 그리드: 썸네일, 제목, 기본 정보, 평점 정렬 옵션: 인기순, 최신순, 평점순, 날짜순 페이지네이션 또는 무한스크롤
윈도우 형식 및 구성	반응형 그리드 레이아웃, 동적 필터링 UI
데이터 형식 및 구성	JSON 배열(검색 결과 목록)
명령 형식	GET 요청(쿼리 파라미터 포함)
종료 메시지	“검색 결과가 없습니다” 또는 결과 개수 표시

Table 3.2: User Interface of Search Results

이름	공연 / 영화 상세 정보 화면
목적 / 내용	선택한 공연/ 영화의 상세 정보 및 예매 링크 제공
입력 주체 / 출력 목적지	사용자 / 서버
범위 / 정확도 / 허용 오차	해당 없음
단위	화면
시간 / 속도	페이지 로드 후 1초 이내 정보 표시
타 입출력과의 관계	KOBIS, TMDB API 호출 및 GPT-4o 요약 생성
화면 형식 및 구성	포스터 이미지(좌측 영역) 기본 정보: 제목, 감독/연출, 출연진, 장르, 기간, 러닝타임 LLM 생성 요약: AI 분석 설명 예매 버튼: 외부 예매 사이트 링크 관련 인물 정보: 감독, 배우의 프로필 카드 유사 추천 작품 세션
윈도우 형식 및 구성	2단 레이아웃(이미지 + 정보), 스크롤 가능
데이터 형식 및 구성	JSON 객체(상세 메타데이터)
명령 형식	GET /performances/:id 또는 /movies/:id
종료 메시지	"정보를 불러올 수 없습니다"(오류 시)

Table 3.3: User Interface of Detail Page

3.1.2 user Interfaces

이름	시스템에서 사용 가능한 디바이스
목적 / 내용	키보드, 마우스를 사용한 사용자의 입력
입력 주체 / 출력 목적지	사용자 / 서버
범위 / 정확도 / 허용 오차	해당 없음
단위	해당 없음
시간 / 속도	사용자의 입력 / 문제 풀이에 해당하는 처리
타 입출력과의 관계	해당 없음
화면 형식 및 구성	해당 없음
윈도우 형식 및 구성	해당 없음
데이터 형식 및 구성	해당 없음
명령 형식	마우스 클릭, 키보드 입력
종료 메시지	해당 없음

Table 3.4: Hardware Interface of Applicable Device for the System

3.1.3 Software Interfaces

이름	웹 브라우저
목적 / 내용	웹 애플리케이션 실행 환경
입력 주체 / 출력 목적지	해당 없음
범위 / 정확도 / 허용 오차	Chrome, Edge, Firefox, Safari와 같은 웹 브라우저에서 사용 가능
단위	해당 없음
시간 / 속도	사용자 입력에 따른 즉각적인 처리
타 입출력과의 관계	해당 없음
화면 형식 및 구성	웹 브라우저를 통한 웹사이트 출력
윈도우 형식 및 구성	해당 없음
데이터 형식 및 구성	HTML, CSS, JavaScript
명령 형식	HTTP / HTTPS 요청
종료 메시지	해당 없음

Table 3.5: Software Interface – Web Browser

이름	외부 API(KOPIS, KOBIS, TMDB)
목적 / 내용	공연 및 영화 정보 조회

입력 주체 / 출력 목적지	백엔드 서버 / 외부 API
범위 / 정확도 / 허용 오차	KOBIS: 박스오피스 및 영화 정보 TMDB: 영화 메타데이터, 포스터, 평점
단위	API 요청
시간 / 속도	응답 시간 평균 500ms 이내
타 입출력과의 관계	백엔드에서 호출 후 프론트엔드로 전달
화면 형식 및 구성	해당 없음
윈도우 형식 및 구성	해당 없음
데이터 형식 및 구성	JSON, XML(API 별 상이)
명령 형식	HTTP GET(API Key 인증)
종료 메시지	HTTP 상태 코드(200, 400, 404, 500 등)

Table 3.6: Software Interface – EXTERNAL APIS

이름	GPT-4o API
목적 / 내용	자연어 처리 및 콘텐츠 추천 생성
입력 주체 / 출력 목적지	백엔드 서버 / OpenAI API
범위 / 정확도 / 허용 오차	자연어 쿼리 이해 및 맞춤형 추천 응답 생성
단위	API 요청
시간 / 속도	응답 시간 평균 2-3초
타 입출력과의 관계	ChromaDB 벡터 검색 결과를 컨텍스트로 활용
화면 형식 및 구성	해당 없음
윈도우 형식 및 구성	해당 없음
데이터 형식 및 구성	JSON(프롬프트 및 응답)
명령 형식	POST /v1/chat/completions(Bearer Token 인증)
종료 메시지	JSON 응답 또는 HTTP 에러 코드

Table 3.7: Software Interface – GPT-4o API

3.1.4 Communication Interfaces

이름	RESTful API 통신
목적 / 내용	프론트엔드와 백엔드 간 HTTP/HTTPS 통신을 위한 데이터 교환. JSON 형식으로 요청 및 응답 데이터 전송
입력 주체 / 출력 목적지	클라이언트 / 백엔드 서버
범위 / 정확도 / 허용 오차	해당 없음
단위	HTTP 요청 / 응답
시간 / 속도	평균 응답 시간 500ms 이내
타 입출력과의 관계	사용자 입력 → API 요청 → 데이터베이스 조회 → 응답 변환
데이터 형식	JSON 요청 / 응답 구조 JWT 토큰 기반 인증 HTTP 상태 코드(200, 400, 404, 500 등)
명령 형식	GET, POST, PUT, PATCH, DELETE, 메서드 ex. GET /api/v1/performances?genre=musical
종료 메시지	JSON 형식 에러 메시지 ex. {"success": false, "error": "NOT_FOUND"}

Table 3.8: Communication Interface – RESTful API

이름	WebSocket 실시간 통신
목적 / 내용	실시간 챗봇 대화 및 알림 전송
입력 주체 / 출력 목적지	클라이언트 / 백엔드 서버
범위 / 정확도 / 허용 오차	해당 없음
단위	메시지
시간 / 속도	실시간 (Lat < 100ms)
타 입출력과의 관계	GPT-4o API와 연동하여 실시간 응답 생성
데이터 형식	이벤트 타입: chat.message, notification.new JSON 메시지 페이로드
명령 형식	WebSocket 이벤트 송수신(wss://)

종료 메시지

연결 종료 이벤트(close event)

Table 3.9: Communication Interface – WebSocket

3.2 Function requirements

3.2.1 Use Case

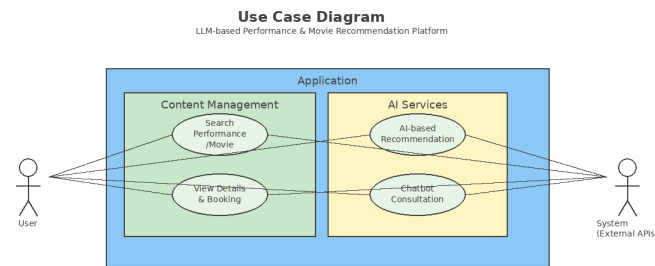
Use Case Name	Searching Performance / Movie
Actor	사용자
Description	사용자가 원하는 영화를 검색하여 관련 정보를 조회하는 프로세스이다.
Normal Course	1. 사용자는 검색창에 원하는 공연명(영화명), 감독명, 배우명 등을 입력한다. 2. 시스템은 KOBIS, TMDB API를 통해 정보를 검색한다. 3. 검색 결과를 리스트 형태로 표시한다. 4. 사용자는 필터(장르, 날짜, 지역, 가격)를 적용시켜 결과를 세분화할 수 있다. 5. 검색결과가 없을 경우 “검색 결과가 없습니다.” 메시지를 표시한다.
Precondition	해당 없음
Post Condition	검색 기록이 저장되어 추후 추천 알고리즘에 활용된다.
Assumptions	외부 API(KOBIS, TMDB)가 정상적으로 작동해야 한다.

Table 3.10: Use Case of Searching Movie

Use Case Name	AI-Based Recommendation
Actor	사용자
Description	GPT-4o 및 ChromaDB를 활용하여 사용자의 취향에 맞는 공연 및 영화를 추천하는 프로세스이다.
Normal Course	사용자가 원하는 장르, 내용 등을 입력하여 추천을 요청한다. ChromaDB에서 유사한 콘텐츠 벡터를 검색한다. GPT-4o API를 호출하여 자연어 기반 추천 설명을 생성한다. 추천 결과를 카드 형식으로 표시하며, 각 추천 항목에는 포스터, 제목, 간단한 설명, 추천 이유가 포함된다.
Precondition	사용자는 충분한 정보를 입력한다.
Post Condition	추천 결과가 저장되어 향후 추천 정확도를 향상시킨다.
Assumptions	GPT-4o API와 ChromaDB가 정상적으로 작동해야 한다.

Table 3.11: Use Case of AI-Based Recommendation

3.2.2 Use Case Diagram



3.2.3 Data Dict



Field	Index	Constraint	Description
content		Not Null	메시지 내용
preferred_genres			선호 장르 목록 (JSON)
created_at		Not Null	메시지 생성 시간
role	Index	Not Null	메시지 주체 (User / AI)

Table 3.12: Chat History

Field	Key	Constraint	Description
id	PK	Not Null	영화 고유 ID (KOBIS ID)
title		Not Null	영화 제목 (KOBIS)
overview		Not Null	줄거리 (TMDB)
posterUrl		Not Null	포스터 이미지 (TMDB)
releaseDate		Not Null	개봉일 (KOBIS)
rating		Not Null	평점 (TMDB)
genres		Not Null	장르 (TMDB)
actors		Not Null	배우 (KOBIS)
directors		Not Null	감독 (KOBIS)
nationName			제작 국가 (KOBIS)

Table 3.14: Movie

Field	Key	Constraint	Description
id	PK	Not Null, Auto Increment	관심 목록 ID
user_id	FK	Not Null	사용자 ID
content_type		Not Null	콘텐츠 유형 (performance/movie)
content_id		Not Null	콘텐츠 ID
added_at		Not Null	추가 시간

Table 3.15: Favorite

3.3 Performance requirements

아래는 본 시스템의 성능 요구사항에 관한 것이다. 예측에 기반한 내용이며, 실제 구현 시에 달라질 수 있다.

3.3.1 Static Numeric Requirements

시스템은 최소 10명의 동시 접속자를 처리할 수 있어야 한다. 데이터베이스는 10GB 이상의 저장 용량을 확보해야 하며, 영화 100,000개의 콘텐츠 정보를 저장할 수 있어야 한다. 시스템은 외부 API(KOBIS, TMDB, GPT-4o) 호출을 일일 최대 10,000회까지 처리할 수 있어야 한다.

3.3.2 Dynamic Numeric Requirements

시스템은 동시에 최소한 10명의 사용자 접속을 유지할 수 있어야 한다. 각 웹페이지는 동시에 최소한 10명의 사용자가 접근할 수 있도록 대역폭과 응답시간을 보장하여야 한다. 그리고 초당 100건 이상의 트랜잭션(TPS)을 처리할 수 있어야 한다.

사용자가 메인페이지 및 검색 결과 페이지를 로딩하는데 걸리는 시간은 2초 이내로 완료되어야 한다. 검색 쿼리에 대한 응답 시간은 평균 1초 이내, 최대 3초 이내로 데이터베이스에 저장되고 이를 나타낼 수 있어야 한다.

GPT-4o API를 통한 AI 추천 및 챗봇 응답은 평균 2-3초, 최대 5초 이내로 표시되어야 한다.

외부 API(KOBIS, TMDB) 호출 응답 시간은 평균 500ms, 최대 2초 이내로 완료해야 한다. 데이터베이스 쿼리 실행 시간은 평균 100ms, 복잡한 쿼리는 500ms 이내로 완료되어야 한다.

WebSocket을 통한 실시간 챗봇 통신의 왕복 지연시간(RTT)은 100ms 이내로 완료해야 한다.

시스템의 에러율은 0.1% 이하 (99.9% 성공률)를 유지해야 하며, 월간 시스템 가용성은 99.5% 이상이어야 한다.

포스터 이미지 로딩 시간은 평균 500ms 이내로 완료되어야 하며, 지연 로딩(Lazy Loading)을 통해 초기 페이지 로딩 성능을 최적화한다.

3.4 Logical database requirements

본 프로젝트의 데이터베이스는 RAG시스템의 핵심 구성 요소로, 사용자의 자연어 쿼리를 효율적으로 처리하기 위한 논리적 구조를 가진다.

3.4.1 Type of data

1) 벡터 데이터 (Vector Data) : 영화의 특정 정보(줄거리, 제목, 감독명, 배우명)을 로컬 임베딩 모델을 이용해 변환한 고차원 벡터이다. 사용자의 쿼리벡터와 '의미/문맥상 유사도'를 비교하는데 사용된다.

2) 메타데이터 (Metadata) : 벡터와 1:1로 매칭되는 원본 정보로 이 정보는 LLM에게 컨텍스트로 제공되거나, 검색 결과를 필터링하는 데 사용된다.

Key	Type	Value
Title	문자열	영화 제목
Director	문자열	감독명
Actors	문자열	주요 배우 목록
Overview	문자열	전체 줄거리
Genres	문자열	장르
Release_date	문자열	개봉일 (ex. "2025-01-01")
Vote_average	숫자 / float	TMDV 평점
Certification	문자열	관람 등급
Kobis_code	문자열	KOBIS 고유 식별자

[Table 1] Type of Metadata

3.4.2 Frequency of use

1) 쓰기 - 매우 낮다. etl 스크립트를 이용해 오프라인에서 일괄적으로 생성하기에 주기적인 업데이트를 제외하면 쓰기 작업이 거의 발생하지 않는다.

2) 읽기 - 매우 높다. 모든 사용자 쿼리는 backend서버를 통해 최소 1회의 데이터베이스 읽기 작업을 수행하기 때문에 시스템은 읽기 작업에 최적화되어야 한다.

3.4.3 Access control

1) ERL(데이터 처리기) : Read and Write. 데이터베이스의 모든 컬렉션에 대한 생성, 수정, 삭제 권한을 가진다.

2) Backend(메인 서버) : Read-Only. 서버는 보안 및 안정성을 위해 DB의 내용을 수정할 수 없으며, 오직 4개의 컬렉션에 대한 검색 및 필터링 권한만 가진다.

3.4.4 Integrity constraints

1) 고유 식별자: kobis_code를 영화의 고유 ID로 사용하여 중복 저장을 방지한다.

2) 데이터 종속성: 모든 데이터는 KOBIS와 TMDB API로부터 수집되므로, 외부 소스의 정보 정확성에 의존한다.

3) 필수 정보: title, overview, kobis_code 및 벡터 값은 NULL이 될 수 없다.

4) 데이터 연관: metadata에 저장된 release_date, vote_average, certification 정보는 backend 서버의 '메타데이터 필터링' 로직과 형식이 반드시 일치해야 한다.

3.5 Design constraints

3.5.1 Physical design constraints

사용자는 데스크톱, 노트북 등의 웹 브라우저에서 이용할 것을 권장한다. 또한 **backend** 서버와 **etl** 스크립트는 **Python** 실행 환경을 필요로 한다. 특히, 로컬 임베딩 모델을 메모리에 로드해야 하므로, 서버는 최소 **4GB**이상의 가용 **RAM**을 확보할 것을 권장한다. 그리고 시스템은 별도 서버 설치가 필요 없는 파일 기반의 벡터 데이터베이스 **ChromaDB**를 이용한다. **ChromaDB**는 메타데이터 관리를 위해 내부적으로 **SQLite**를 사용하므로, **backend** 서버가 **Data**폴더에 직접 접근(읽기)할 수 있어야 한다.

3.5.2 User class

시스템의 **backend**는 **Typescript** 기반의 **Nest.js** 프레임워크를 사용하여 개발하며, 모든 **backend** **Python** 코드는 **PEP 8** 코딩 표준을 준수한다. 그리고 **Python** 변수 및 함수 이름은 **snake_case** 를, 클래스 이름은 **PascalCase**를 따른다. **frontend**와 **backend** 간의 모든 **API** 통신은 **JSON** 표준 데이터 형식을 따른다.

3.6 Software system attributes

3.6.1 Reliability

LLMuse 시스템은 외부 **API** 호출 실패 시 재시도 로직을 적용하여 일시적인 오류에 대응하며, 재시도 실패나 **Rate Limiting** 발생 시에는 사용자에게 명확한 오류 메시지 또는 캐시된 데이터를 제공한다. 사용자 질의에 대한 답변 생성 시간은 **1분** 이내를 목표로 하며, **KOBIS-TMDB** 간 영화 제목 매칭 실패와 같은 데이터 불일치 상황에서는 백엔드 콘솔에 로그를 출력하여 오류 지점을 명확히 식별할 수 있도록 한다. 모든 예외 상황 발생 시 서버는 구체적인 오류 메시지를 반환하며, 비핵심 기능의 장애가 발생하더라도 핵심 기능이 중단되지 않도록 부분적 복구 방식을 유지한다.

3.6.2 Availability

서비스는 예측 가능한 시간대에 안정적으로 이용 가능함을 목표로 한다. 외부 **API(OpenAI, KOBIS, TMDB)** 장애 발생 시에는 캐시된 데이터를 활용하거나 제한적인 서비스를 제공하며, 사용자에게 명확한 오류 메시지를 통해 상태를 안내한다. 데이터 수집 및 업데이트 과정에서 서비스 일시 중단이 발생할 수 있으며, 이 경우 관리자 또는 사용자에게 사전 공지를 제공한다. 영화 데이터 업데이트는 일괄 처리 형태로 수행되며, 데이터베이스의 안정성과 무결성을 최우선으로 고려한다.

3.6.3 Security

KOBIS, TMDB, OpenAI API Key는 **.env** 파일을 통해 환경 변수 형태로 관리하며, 클라이언트 코드에 노출되지 않도록 서버 측에서만 접근 가능하도록 한다. 모든 데이터 통신은 기본적으로 **HTTPS** 프로토콜을 사용하도록 설정되며, 사용자 질의 로그 저장 시 개인 식별 정보를 포함하지 않는 형태로 저장하여 데이터를 보호한다.

3.6.4 Maintainability

명확한 구조와 코딩 컨벤션을 통해 팀원 간의 협업 및 기능 수정/추가 용이하게 한다. **Frontend(Next.js) - Backend(Nest.js) - VectorDB(ChromaDB)**의 역할 분리를 명확히 하는 모듈화된 아키텍처를 기반으로 한다. **TypeScript**를 통한 타입 안정성을 확보하며, 주요 기능 로직에는 코드 주석을 상세히 달고 **README** 파일을 통해 배포 방법 등을 명확히 설명하는 문서화를 수행한다. 테스트 전략으로는 주요 기능에 대한 기능 테스트를 수동으로 수행하여 핵심 동작의 정상 여부를 검증한다.

3.6.5 Portability

표준 기술을 사용하고 배포 환경 의존성을 낮추어 이식성을 확보한다. RESTful API를 통해 프론트엔드와 백엔드의 교체가 용이한 느슨한 결합 구조를 유지한다. 데이터베이스 접근 로직을 분리하는 간단한 **Repository** 패턴을 적용하여 ChromaDB 외 다른 DB로의 교체 시에도 최소한의 코드 수정으로 대응하도록 설계된다.